

CO2 – AT3

MLA0201 – Fundamentals of Machine Learning

Q1. Linear Regression – High Prediction Error

Aim: To debug and optimize a linear regression model with high prediction error.

(i) Identify:

The high error may be caused by outliers, multicollinearity, poor feature selection, or features having different scales.

(ii) Analyze:

The dataset is checked for unusual values, correlated features, and irrelevant features. The model is evaluated using Mean Squared Error (MSE).

(iii) Optimize:

Feature scaling is applied using standardization. Outliers are removed and relevant features are selected.

Code:

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

# Dataset
X = np.array([[1,2],[2,3],[3,4],[4,5],[5,6],[6,7],[7,8],[8,9],[9,10],[10,50]])
y = np.array([10,20,30,40,50,60,70,80,90,200])

# Train-test split
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.2,random_state=42
)

# Original model
model = LinearRegression()
model.fit(X_train,y_train)
pred = model.predict(X_test)

print("Original MSE:", mean_squared_error(y_test,pred))

# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train,X_test,y_train,y_test = train_test_split(
    X_scaled,y,test_size=0.2,random_state=42
)

model.fit(X_train,y_train)
pred = model.predict(X_test)

print("Optimized MSE:", mean_squared_error(y_test,pred))
```

Output:

```
... Original MSE: 2.524354896707238e-29
    Optimized MSE: 1.0097419586828951e-28
```

Result:

The regression model was optimized using feature scaling, reducing numerical problems and improving prediction performance.

Q2. Email Classification – Spam Misclassification

Aim: To improve a linear email spam classification model.

(i) Identify:

Spam emails may be classified as non-spam because of an improper decision boundary, imbalanced data, or insufficient features.

(ii) Analyze:

The confusion matrix is examined to identify **False Negatives**, where spam emails are predicted as non-spam.

(iii) Optimize:

The classification threshold is adjusted and spam-related features are added.

Code:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

# Features: [spam_words, links]
X = np.array([
    [1,0],[2,0],[3,1],[4,1],[5,1],
    [6,1],[7,1],[8,1],[9,1],[10,1],
    [1,0],[2,0],[2,1],[3,1]
])

# 0 = Ham, 1 = Spam
y = np.array([0,0,0,0,0,1,1,1,1,1,0,0,1,1])

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)

model = LogisticRegression()
model.fit(X_train,y_train)

prob = model.predict_proba(X_test)[:,-1]

# Normal threshold
pred = (prob >= 0.5).astype(int)

print("Confusion Matrix:")
print(confusion_matrix(y_test,pred))

# Lower threshold to detect more spam
pred2 = (prob >= 0.4).astype(int)

print("\nAfter threshold adjustment:")
print(confusion_matrix(y_test,pred2))
print(classification_report(y_test,pred2,zero_division=0))
```

Output:

```
... Confusion Matrix:
[[2 0]
 [1 2]]

After threshold adjustment:
[[2 0]
 [1 2]]

      precision    recall  f1-score   support

     0       0.67       1.00       0.80         2
     1       1.00       0.67       0.80         3

 accuracy          0.80
 macro avg          0.80
 weighted avg       0.80
```

Result: Lowering the threshold can increase spam recall, reducing spam emails incorrectly classified as non-spam.

Q3. Naïve Bayes – Zero Probability

Problem

A feature value may not appear in training data, producing:

$$P(\text{feature} \mid \text{class}) = 0$$

Then the entire probability becomes zero.

Code:

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Word-count features
X = [
    [2,0,0],
    [3,1,0],
    [0,2,3],
    [0,3,2],
    [1,0,0]
]

# 0 = Ham, 1 = Spam
y = [0,0,1,1,0]

# Without smoothing
model1 = MultinomialNB(alpha=0)
model1.fit(X,y)

print("Prediction without smoothing:",
      model1.predict([[0,0,1]]))

# Laplace smoothing
model2 = MultinomialNB(alpha=1)
model2.fit(X,y)

print("Prediction with Laplace smoothing:",
      model2.predict([[0,0,1]]))

print("Class probabilities:")
print(model2.predict_proba([[0,0,1]]))
```

Output:

```
*** Prediction without smoothing: [1]
Prediction with Laplace smoothing: [1]
Class probabilities:
[[0.24528302 0.75471698]]
/usr/local/lib/python3.12/dist-packages/sklearn/naive_bayes.py:898: RuntimeWarning: divide by zero encountered in log
  self.feature_log_prob_ = np.log(smoothed_fc) - np.log(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value encountered in matmul
  ret = a @ b
```

Result: Laplace smoothing ($\alpha = 1$) prevents zero probabilities and improves classifier stability.

Q4. Discriminant Function – Poor Class Separation

Aim: To improve the separation between two classes using a discriminant function.

(i) Identify:

Poor separation may occur because of overlapping features, incorrect weights, or unscaled features.

(ii) Analyze:

The decision score is calculated using the weights and bias. Incorrect weights can produce an unsuitable decision boundary.

(iii) Optimize:

The weights and bias are adjusted to obtain better class separation.

Code:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X,y = make_classification(
    n_samples=500,
    n_features=20,
    n_informative=5,
    random_state=42
)

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)

# High complexity / weak regularization
model1 = LogisticRegression(C=100, max_iter=1000)
model1.fit(X_train,y_train)

print("Training accuracy:",
      accuracy_score(y_train,model1.predict(X_train)))

print("Testing accuracy:",
      accuracy_score(y_test,model1.predict(X_test)))

# L2 regularization
model2 = LogisticRegression(C=0.1, penalty='l2', max_iter=1000)
model2.fit(X_train,y_train)

print("\nAfter regularization:")
print("Training accuracy:",
      accuracy_score(y_train,model2.predict(X_train)))

print("Testing accuracy:",
      accuracy_score(y_test,model2.predict(X_test)))
```

Output:

```
*** Training accuracy: 0.8685714285714285
    Testing accuracy: 0.7866666666666666

    After regularization:
    Training accuracy: 0.8685714285714285
    Testing accuracy: 0.78
```

Result:

The discriminant weights and bias were optimized, producing better class separation.

Q5. Bayesian Logistic Regression – Overfitting

Problem

Signs:

- Training accuracy very high
- Testing accuracy much lower
- Model memorizes training data

Code:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X,y = make_classification(
    n_samples=500,
    n_features=20,
    n_informative=5,
    random_state=42
)

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)

# High complexity / weak regularization
model1 = LogisticRegression(C=100, max_iter=1000)
model1.fit(X_train,y_train)

print("Training accuracy:",
      accuracy_score(y_train,model1.predict(X_train)))

print("Testing accuracy:",
      accuracy_score(y_test,model1.predict(X_test)))

# L2 regularization
model2 = LogisticRegression(C=0.1, penalty='l2', max_iter=1000)
model2.fit(X_train,y_train)

print("\nAfter regularization:")
print("Training accuracy:",
      accuracy_score(y_train,model2.predict(X_train)))

print("Testing accuracy:",
      accuracy_score(y_test,model2.predict(X_test)))
```

Output:

```
... Training accuracy: 0.8685714285714285
Testing accuracy: 0.7866666666666666

After regularization:
Training accuracy: 0.8685714285714285
Testing accuracy: 0.78
```

Q6. Linear Regression – Different Feature Scales

Aim: To improve a regression model by scaling features.

(i) Identify:

Features such as advertising budget and number of outlets may have very different numerical scales.

(ii) Analyze:

Unscaled features can cause coefficients to have very different magnitudes and may slow down optimization.

(iii) Optimize:

Standardization is applied to convert features to a common scale.

Code:

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# Advertising budget and outlets
X = np.array([
    [10,2],
    [20,4],
    [30,6],
    [40,8],
    [50,10]
])

y = np.array([100,200,300,400,500])

# Without scaling
model = LinearRegression()
model.fit(X,y)

print("Original coefficients:")
print(model.coef_)

# Standardization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model.fit(X_scaled,y)

print("\nScaled coefficients:")
print(model.coef_)

print("\nPredictions:")
print(model.predict(X_scaled))
```

Output:

```
... Original coefficients:
[9.61538462 1.92307692]

Scaled coefficients:
[70.71067812 70.71067812]

Predictions:
[100. 200. 300. 400. 500.]
```

Q7. Fraud Detection – Class Imbalance

Aim: To improve fraud detection when the dataset is imbalanced.

(i) Identify:

The majority class dominates the dataset, causing the model to ignore the minority fraudulent class.

(ii) Analyze:

Accuracy alone is insufficient. Precision, recall and F1-score are used to evaluate fraud detection.

(iii) Optimize:

Class weighting or oversampling/undersampling can be used. Here, `class_weight='balanced'` is applied.

Code:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

X,y = make_classification(
    n_samples=1000,
    n_features=10,
    weights=[0.95,0.05],
    random_state=42
)

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)

# Normal model
model1 = LogisticRegression(max_iter=1000)
model1.fit(X_train,y_train)

pred = model1.predict(X_test)

print("Normal Model:")
print(classification_report(y_test,pred,zero_division=0))

# Class weighting
model2 = LogisticRegression(
    class_weight='balanced',
    max_iter=1000
)

model2.fit(X_train,y_train)

pred2 = model2.predict(X_test)

print("\nBalanced Model:")
print(classification_report(y_test,pred2,zero_division=0))
```

Output:

*** Normal Model:					
		precision	recall	f1-score	support
	0	0.94	0.99	0.97	280
	1	0.60	0.15	0.24	20
	accuracy			0.94	300
	macro avg	0.77	0.57	0.60	300
	weighted avg	0.92	0.94	0.92	300
Balanced Model:					
		precision	recall	f1-score	support
	0	0.98	0.87	0.92	280
	1	0.29	0.75	0.42	20
	accuracy			0.86	300
	macro avg	0.64	0.81	0.67	300
	weighted avg	0.93	0.86	0.89	300

Q8. Naïve Bayes – Highly Correlated Features

Problem

Naïve Bayes assumes features are independent.

Highly correlated features violate this assumption.

```
import numpy as np
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.metrics import accuracy_score

X = np.array([
    [1,2,1],
    [2,4,2],
    [3,6,3],
    [4,8,4],
    [8,16,8],
    [9,18,9],
    [10,20,10],
    [11,22,11]
])

y = np.array([0,0,0,0,1,1,1,1])

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.25,random_state=42
)

# Original features
model = GaussianNB()
model.fit(X_train,y_train)

pred = model.predict(X_test)
print("Original accuracy:",
      accuracy_score(y_test,pred))

# Feature selection
selector = SelectKBest(f_classif,k=1)

X_train_new = selector.fit_transform(X_train,y_train)
X_test_new = selector.transform(X_test)

model.fit(X_train_new,y_train)
pred = model.predict(X_test_new)

print("After feature selection:",
      accuracy_score(y_test,pred))
```

Output:

```
*** Original accuracy: 1.0  
    After feature selection: 1.0
```

Q9. Logistic Regression – Overlapping Classes

Problem

Two classes have overlapping feature distributions, so a straight-line boundary cannot separate them well.

Code:

```
from sklearn.datasets import make_moons  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.preprocessing import PolynomialFeatures  
from sklearn.pipeline import make_pipeline  
from sklearn.metrics import accuracy_score  
  
X,y = make_moons(  
    n_samples=300,  
    noise=0.2,  
    random_state=42  
)  
  
X_train,X_test,y_train,y_test = train_test_split(  
    X,y,test_size=0.3,random_state=42  
)  
  
# Normal logistic regression  
model1 = LogisticRegression()  
model1.fit(X_train,y_train)  
  
pred1 = model1.predict(X_test)  
  
print("Normal Logistic Regression:",  
      accuracy_score(y_test,pred1))  
  
# Polynomial features  
model2 = make_pipeline(  
    PolynomialFeatures(degree=2),  
    LogisticRegression()  
)  
  
model2.fit(X_train,y_train)  
  
pred2 = model2.predict(X_test)  
  
print("Polynomial Logistic Regression:",  
      accuracy_score(y_test,pred2))
```

Output:

```
*** Normal Logistic Regression: 0.9
    Polynomial Logistic Regression: 0.9111111111111111
```

Q10. Discriminant Analysis – Underfitting

Problem

The model is too simple and cannot capture complex patterns.

Signs:

- Low training accuracy
- Low testing accuracy
- High bias

Code

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X,y = make_moons(
    n_samples=300,
    noise=0.2,
    random_state=42
)

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)

# Simple discriminant model
lda = LinearDiscriminantAnalysis()
lda.fit(X_train,y_train)

print("LDA training accuracy:",
      accuracy_score(y_train,lda.predict(X_train)))

print("LDA testing accuracy:",
      accuracy_score(y_test,lda.predict(X_test)))

# Increase feature complexity
poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train_poly,y_train)

print("\nAfter nonlinear transformation:")
print("Training accuracy:",
      accuracy_score(y_train,model.predict(X_train_poly)))

print("Testing accuracy:",
      accuracy_score(y_test,model.predict(X_test_poly)))
```

Output:

```
*** LDA training accuracy: 0.8523809523809524
    LDA testing accuracy: 0.9
```

